
Reinforcement Learning Assignment 1

Manuel Pérez Belizón 4416880

Abstract

Tabular Reinforcement Learning algorithms work well on small environments but they struggle with bigger problems. This limitation can be solved using Neural Networks to approximate the value function, this is the main focus of this assignment, in particular we will work on the Deep Q-Network(DQN) algorithm an algorithm that tries to solve the limitation of the Q-table in the Q-learning algorithm by approximating the Q-values using a Neural Network. The assignment includes experiments demonstrating the effectiveness of DQN dealing with the game Cartpole. Results highlight the importance of hyperparameter tuning and network architecture in achieving optimal performance.

1. Introduction

One of the fundamental RL algorithms is Q-learning, which uses a Q-value table to estimate the expected future rewards of actions. However, in high-dimensional state spaces environments, maintaining a Q-table can become impossible. To address this, Deep Q-Networks (DQN) leverage deep learning to approximate the Q-values using a neural network. But using deep neural networks for Q-learning introduces three main problems: coverage, correlation and convergence.

The objective of this coursework is to explore the basics of DQN. In the first part of this assignment, we will try to understand the theoretical principles of said algorithm and in the second part we will implement a version of this algorithm and run several experiments.

2. Theory

First of all, we will give a small notion about Neural Networks. Neural Networks are models that produce an output given an input. The input propagates through the network where it is through weighted connections and activation functions. Then the network calculates the error of the prediction using a loss function and learns by propagating backwards this error adjusting the weights of each layer with gradient descent or other mechanism if needed. The main goal of this algorithm is to minimize this loss function

improving the predictions.

2.1. Challenges

DQN is an adaptation of the Q-Learning algorithm, the adaptation comes with three main challenges that need to be solved:

- Coverage: in the Q-Learning algorithm covering the whole state space guaranties convergence to an optimal solution, this is not possible in DQN as we are treating with high dimensional problems.
- Correlation: in DQN all subsequent states differ only in one feature therefore these states are correlated this results in low coverage and biased training, reducing convergence towards the optimal Q-function, increasing the probability of local optima and feedback loops.
- Convergence: in DQN the target Q-values for learning are not fixed, they dynamically change during the training. Since both the predicted and the target depend on the same parameters, updates can become unstable leading to divergence.

2.2. Solutions

Two solutions can be applied to make the algorithm stable, these are important as they will be implemented and discussed later.

- Experience Replay(ER): in this technique we introduce a buffer that stores previous states, when training this states will be randomly sampled breaking the correlation of only taking the subsequent states.
- Target Network(TN): in this technique the network is cloned every some updates, this cloned network, the target network, will be used to generate the targets for the later updates, this makes the weights of the target network change much slower improving stability of the Q-targets.

2.3. Loss Function

The loss function for DQN derives from the Q-learning update formula, which derives from Bellman equation but in

DQN two networks with the same architecture are used, one for the estimation of the target and the other for the estimation of the prediction, where the result of the target is taken as ground truth for the prediction. So for the loss function we use the square difference between the Q-value(target network) and the old update target (prediction network) and the formula looks like:

$$\mathbb{L} = (Q_{\Theta_t}(s, a) - r + \gamma \max_{a'} Q_{\Theta_{t-1}}(s', a'))^2 \quad (1)$$

We cannot use update rules as in tabular RL because several reasons:

1. The update rules in tabular RL are made for direct updates to the table exactly once every state making them independent while in DQN an update to one Q-value could alter many others.
2. With a tabular direct update there wouldn't be the possibility of backpropagation which is needed for learning in DQN.
3. The update rule in tabular RL exact values while in DQN we use an approximate function.

2.4. DQN Pseudo Code

Now we are providing the pseudo code for the DQN algorithm with the ER and TN mechanisms:

Algorithm 1 DQN

```

1: Initialize replayBuffer
2: Initialize Q and Qt networks
3: for i = 1 to N episodes do
4:   set initial state
5:   for each step in episode do
6:     action = greedySelection
7:     reward = computeReward
8:     replayBuffer.
       append(action, reward, state, nextState)
9:     sample minibatch from replayBuffer
10:    compute Qtarget using Qt
11:    loss = (Qtarget - Q(s, a))2
12:    gradient descent on loss
13:    update Q with backpropagation
14:    update Qt after M steps
15:   end for
16: end for=0
    
```

3. Experiments

Now that the theory is clear, in this section we will implement a Python DQN algorithm for the Cartpole environment, first without ER and TN (Naive) and then we will add each of the methods separated and together.

We will use Gymnasium and Torch for the implementations.

3.1. Naive DQN

3.1.1. NEURAL NETWORK

First we have to implement a Neural Network to approximate the values, we will start building a very simple Neural Network.

The first layer of the Neural Network, the input layer, should have as many neurons as the environment observation space, in the case of Cartpole is four that correspond to cart position, cart velocity, pole angle and pole velocity.

Then we will have one fully connected hidden layer with 64 neurons and a ReLu activation function.

Lastly, in the output layer we must have as many neurons as possible actions therefore in this example we will have 2 neuron in the output layer.

This simple NN might be enough for our problem as the Cartpole problem is quite simple with only 2 actions and 4 observations.

3.1.2. SELECTING ACTION

For selecting the action we will use an epsilon-greedy action selection to balance exploration and exploitation:

$$action = \begin{cases} random_action & \text{if } p < \epsilon \\ best_action & \text{otherwise} \end{cases}$$

where p is a random probability and ϵ is a hyperparameter that will change for every episode.

3.1.3. TRAINING LOOP

The only difference with the 1 algorithm is that in this implementation we are using a Naive approach therefore we don't have a replay buffer and the loss function the Q-values and Q-target from the same Q net.

The loop is run for $num_episodes = 1000$ where each episode finishes either if the goal is failed or when the episode length is greater than 500 steps therefore having 2500 episodes will get us around 500000 total steps.

Also for this an every other configuration we are doing 5 different runs to be sure that the results are not based on luck.

3.1.4. RESULTS

The results for the Naive DQN, 5 runs and 1000 episodes:

From the results shown in Figure 1 we can see that the network is extremely unstable but even with that much instability, it is training and it is getting better results. This

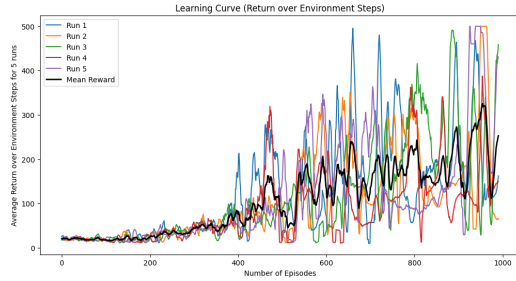


Figure 1. DQN Naive Results

results show that our algorithm is still far from optimal performance.

3.2. Ablation study

For this experiment we will focus on 4 different hyperparameters:

1. Learning rate: it controls how much the weights change on each learning step, smaller learning rates make the learning more stable but can lead stuck the algorithm while bigger learning rates make the training faster but more unstable.
2. Update-to-data ratio: it controls how often the model updates, a smaller ratio will make more updates leading to heavier computation and are more unstable while a bigger ratio makes less updates can help with stability by breaking correlation.
3. Network size: it defines the size of our model's hidden layer (number of neurons), bigger network size can capture more complex state-action relationships at the cost of slower training.
4. Exploration factor: it controls how fast the model shifts from exploration to exploitation, a big factor does more exploration but can make it harder to converge while a small factor exploits more and can get stuck in suboptimal policies.

For each of the hyperparameters we will be trying 3 different values:

- Learning rate: $1e^{-5}$, $1e^{-4}$, $1e^{-3}$
- Update-to-data ratio: 1, 5, 10
- Network size: 32, 64, 128 neurons
- Exploration factor: 0.985, 0.99, 0.995

For this study we are trying every combination but only for 500 episodes as we are going to try a total of 81 possible

combinations and we are going to measure the mean reward obtained for each of the combinations.

The best combination was $learningRate = 1e^{-3}$, $updateRatio = 1$, $networkSize = 128$ and $explorationRatio = 0.99$ achieved a mean reward of 140.217, from the results we can see that the network is still very unstable but it seems that the most relevant hyperparameters are learning rate and exploration factor, in our experiment small learning rates struggle a lot getting out of local minima, big exploration factor makes our network not be able to converge while the smallest exploration factor makes the network to almost stop exploring at only 100 episodes often leading the algorithm to get stuck.

Network size does not seem to have a very big impact, this makes sense as the problem we are solving is not very complex.

From now on the rest of the experiments will use the combination obtained in this experiment.

3.3. Adding ER and TN

In the implementation with ER we add the replay buffer with a certain memory size then inside the training loop, we first append the state, the action, the reward and the next state to the buffer then if we have enough samples inside the buffer we randomly sample a minibatch from the buffer so instead of training one transition per step we use these minibatches.

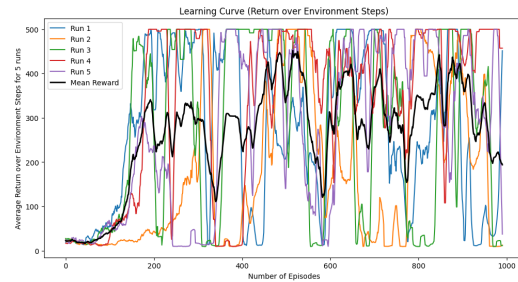


Figure 2. DQN with ER Results

From the Figure 2 we can already see a very big improvement, now all of the runs get to the maximum score in many consecutive episodes, there's a clear learning curve from the episodes 1 to 200 where the runs start achieving the maximum return but from there we can also see that the algorithm struggles and it becomes a bit unstable through the rest of the episodes.

In the implementation with TN only, we introduce the target network which we initialize copying our Q network, we can set this network to evaluation mode as we are not going to train it, we will only train our Q network.

In the training loop, we will update the target network every *target_update_freq* episodes, we use the target network to compute the target values for the loss.

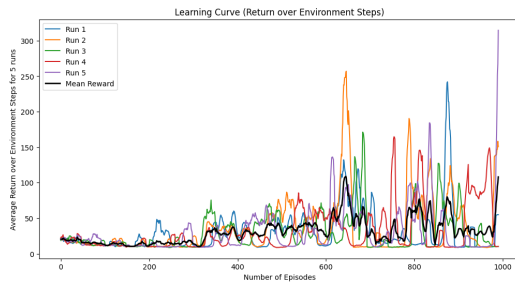


Figure 3. DQN with TN Results

From Figure 3.3 we see that the results of adding the TN mechanism to the DQN does not improve the performance of the algorithm, in fact, it may look like the algorithm is performing worse than before, although the performance is not better we can see that comparing to the results of our Naive DQN that these results are much more stable.

These findings may have happened because of the nature of adding TN where now the model is learning slower so it is possible that the algorithm needs more episodes to converge, decreasing the target update frequency variable could also help training converging faster at the cost of stability, this variable is set to 10 for this experiments which means that it will update the target network at a maximum of 5000 environment steps.

Now for the last experiment we implement a DQN with ER and TN, for this experiment we merge the last to ending up with an algorithm that uses a replay buffer and minibatch sampling and also a target network to compute the target values.

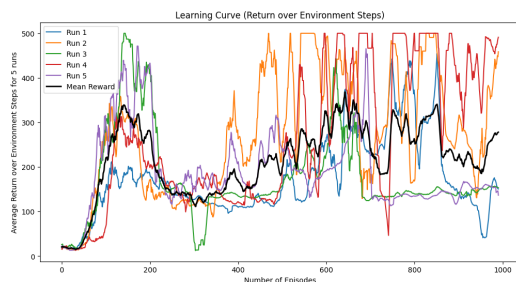


Figure 4. DQN with ER and TN Results

The results for this last experiment shown in Figure 4 give us a good balance between stability and reward performance.

4. Discussion

From the experiments we can observe certain strengths and weaknesses of the different variations of the DQN algorithm.

The Naive DQN showed bad performance and stability overall, this was the expected behavior as this algorithm without ER relies on correlated consecutive states that reduces convergence and without TN the target Q-values change dynamically while training leading to a big instability.

Introducing ER to the algorithm greatly improved its performance, storing past states and randomly sampling them prevented correlation between consecutive states which lead to the improvement of performance. But while the performance improved, the instability of the algorithm was still quite big as the Q-values were still updated using the same network that was actively being trained.

On the other hand, introducing TN improved stability by a lot but by the cost of performance because the target network was only updated periodically, the agent's learning process became slower and less responsive to new information.

Finally, the combination of ER and TN provided the best balance between stability and performance.

In summary, ER improves performance but introduces some instability, TN improves stability but reduces performance, and combining ER and TN provides a well-balanced trade-off between both factors.

5. Conclusion

Through this assignment, we explored the fundamental principles of Deep Q-Networks (DQN) both from a theoretical and experimental perspective. We started with the Naive DQN implementation and progressively incorporated key improvements, namely Experience Replay (ER) and Target Networks (TN), to analyze their impact on learning stability and performance.

Our experiments demonstrated that while the Naive DQN struggles with both stability and performance, introducing ER significantly improves learning efficiency but can lead to fluctuations. Conversely, TN enhances stability but slows down learning. The combination of ER and TN provided a well-balanced solution, though performance was slightly lower than using ER alone.